

OpenAI API를 활용한 텍스트-음성 변환 (TTS) 튜토리얼

OpenAI의 강력한 API를 이용하면 입력한 텍스트를 자연스러운 음성으로 변환할 수 있습니다. 이 튜토리얼에서는 Jupyter Notebook에서 실행할 수 있는 Python 코드와 함께 OpenAI의 텍스트-음성 변환(Text-to-Speech, TTS) 기능을 단계별로 설명합니다. 실습을 통해 API 키 설정부터 기본적인 TTS 사용법, 고급 기능 활용, 그리고 생성된 오디오를 저장하고 재생하는 방법까지 알아보겠습니다.

주요 학습 내용:

1. 환경 설정: OpenAI API 키를 안전하게 저장하고 로드하는 방법 (python-dotenv 활용)
2. 기본 TTS 변환: 간단한 텍스트를 음성으로 변환하는 예제 코드
3. 고급 기능 활용: 음성 모델 선택, 음성 목소리(voice) 변경, 언어 지원 및 속도 조절 등 추가 기능
4. 오디오 파일 저장 및 재생: 생성된 음성을 파일로 저장하고 Jupyter Notebook에서 재생하는 방법

각 섹션에서는 관련 코드 블록과 함께 자세한 설명을 제공하니, 코드를 직접 실행해보며 따라 할 수 있습니다.

1. 환경 설정

Python 코드에서 python-dotenv로 .env 파일을 불러온 뒤, openai.OpenAI() 클래스를 이용해 API 클라이언트 인스턴스를 생성합니다. 이때 API 키는 명시적으로 전달하거나 환경 변수 OPENAI_API_KEY가 설정되어 있어야 합니다. 환경 변수 또는 인자로 API 키를 지정하지 않으면 OpenAI 라이브러리는 다음과 같은 오류를 발생시킵니다:

> OpenAIError: The api_key client option must be set either by passing api_key to the client or by setting the OPENAI_API_KEY environment variable

아래는 환경 로드 및 클라이언트 생성 예제입니다:

위 코드에서는 .env에서 불러온 API 키로 client 객체를 생성했습니다. 이제 이 client를 통해 DALL-E를 비롯한 OpenAI API 요청을 보낼 수 있습니다. (참고로, api_key를 생략하면 OPENAI_API_KEY 환경 변수를 자동으로 참조합니다.)

2. 기본 TTS 변환: 텍스트를 음성으로 간단히 만들기

환경 설정이 완료되었다면, 이제 텍스트를 음성으로 변환해보겠습니다. OpenAI의 TTS API는 입력 텍스트를 받아 사람이 말하는 것 같은 음성 오디오 데이터를 반환합니다. 기본적으로 `**tts-1**`이라는 모델을 사용하며, 음성의 종류(voice)를 선택할 수 있습니다. 우선 간단한 예제로 기본 모델과 한 가지 목소리를 이용해 동작을 확인해보겠습니다.

- <https://developers.openai.com/api/docs/guides/text-to-speech>

위 코드에서는 `openai_client.audio.speech.create(...)` 메소드를 호출하여 TTS 요청을 보냈습니다. 주요 파라미터는 다음과 같습니다:

- `model="tts-1"`: OpenAI의 기본 TTS 모델을 지정합니다. `tts-1` 모델은 실시간 응답에 최적화된 모델로, 비교적 빠르게 결과를 반환합니다.
- `voice="nova"`: 출력 음성의 목소리를 지정합니다. OpenAI TTS에는 여러 프리셋 목소리가 있는데, 여기서는 "nova"라는 목소리를 선택했습니다. (목소리 종류에 대해서는 다음 섹션에서 자세히 다룹니다.)
- `input=text`: 변환할 텍스트 문자열을 입력으로 전달합니다.

`create` 메소드를 호출하면 OpenAI API에 요청을 보내고, 음성 오디오 데이터를 응답으로 받습니다. `response` 객체에는 이진 형식의 오디오 데이터가 포함되어 있습니다. `print(type(response))`로 객체 타입을 출력해보면, OpenAI 라이브러리의 내부 응답 객체임을 확인할 수 있습니다.

이 `response`로부터 실제 오디오 바이너리 데이터를 추출하려면 `response.content` 속성을 사용할

수 있습니다. 예를 들어:

위와 같이 `response.content`를 사용하면 음성 데이터의 바이너리를 얻을 수 있고, `len(audio_data)`로 바이트 크기를 확인할 수 있습니다. 이제 텍스트가 음성 데이터로 변환된 것을 확인했으니, 다음 단계에서 다양한 고급 기능을 활용하는 방법을 알아보겠습니다.

주의: 이 시점까지는 메모리 상에 음성 데이터가 있을 뿐, 실제 소리로 들어보지는 않았습니다. `response.content`를 파일로 저장하거나 플레이어를 통해 재생해야 비로소 음성을 들을 수 있습니다. 오디오 데이터 저장과 재생 방법은 튜토리얼 뒷부분(섹션 4)에서 다룹니다.

3. 고급 기능 활용: 모델 선택, 목소리 변경, 언어 지원 등

OpenAI의 TTS API는 기본 기능 외에도 다양한 옵션을 제공합니다. 이 섹션에서는 고급 기능으로 다음을 다뤄봅니다:

- 다양한 목소리(voice) 선택: 남성/여성 및 톤이 다른 여러 프리셋 음성을 사용할 수 있습니다.
- 모델 선택: 기본 모델 `tts-1` 외에 고품질 모델 `tts-1-hd` 사용법
- 다국어 지원: 영어 이외의 언어로도 TTS가 가능 (한국어, 스페인어 등)
- 음성 속도 조절: `speed` 파라미터로 발음 속도를 빠르게 또는 느리게 조절

각 기능별로 어떻게 사용되는지 예제를 통해 살펴보겠습니다.

3.1 여러 가지 음성 목소리 선택하기

OpenAI TTS에는 다양한 **프리셋 목소리(voice)**가 준비되어 있어 텍스트를 여러 스타일의 음성으로 들을 수 있습니다. 초기에는 6가지 목소리를 제공하며, 각 목소리는 고유한 음색과 톤을 가지고 있습니다. 제공되는 대표적인 음성 이름은 다음과 같습니다:

- Alloy – (예상: 남성적이고 부드러운 음색)
- Echo – (맑고 청아한 톤)
- Fable – (동화 구연 같은 따뜻한 톤)

- Nova – (친근한 여성 음색으로 추정)
- Onyx – (깊고 진중한 남성 음색)
- Shimmer – (밝고 생동감 있는 여성 음색)

각 목소리는 영어를 기반으로 최적화되어 있지만, 다국어 텍스트도 발음할 수 있습니다. 원하는 분위기나 성별 느낌에 따라 위 이름들 중 선택하면 됩니다. 사용 방법은 간단히 voice 파라미터에 해당 이름 문자열을 지정하는 것입니다. 예를 들어, 앞서 기본 예제에서는 "nova" 음성을 사용했는데, 다른 음성으로 바꾸어 출력해보겠습니다:

위 코드에서는 voice="onyx"로 설정하여 Onyx 목소리로 텍스트를 음성 변환했습니다. response_diff_voice.content에 변환된 오디오 데이터가 들어있으며, 바이트 크기를 출력하여 데이터가 생성되었음을 확인했습니다. 나중에 오디오를 재생해보면, 처음 사용한 Nova 목소리와 Onyx 목소리가 다르게 들리는 것을 알 수 있습니다.

>Tip: 음성 이름은 대소문자를 구분하지 않는 소문자로 입력합니다. 또한, OpenAI에서 새로운 목소리를 추가로 제공할 수 있으므로, 최신 정보는 OpenAI 공식 문서나 발표를 참고하세요. (예: 추가로 Ash, Coral, Sage 등의 목소리가 제공되기 시작할 수 있습니다.)

3.2 TTS 모델 선택: 실시간 vs 고품질

OpenAI는 현재 두 가지 TTS 모델 **tts-1**과 **tts-1-hd**를 제공합니다. 기본 모델 tts-1은 응답 속도가 빠른 것이 장점이고, tts-1-hd 모델은 보다 고품질의 음성을 생성하지만 응답이 약간 느릴 수 있습니다. 응답 속도와 음질 간의 트레이드오프가 있으므로, 용도에 맞게 선택하면 됩니다. 예컨대, 실시간 상호작용이 중요하다면 tts-1을, 음성 품질이 최우선이라면 tts-1-hd를 쓰는 식입니다.

모델을 바꾸는 방법은 model 파라미터에 이름을 달리 주는 것입니다. 다음은 고품질 모델 tts-1-hd를 사용한 예제입니다 (음성은 예시로 Alloy를 선택):

위 코드는 tts-1-hd 모델과 Alloy 음성을 사용하여 텍스트를 변환합니다. 겉보기에는 이전과 동일하게 response_hd.content로 오디오 데이터를 얻지만, 실제 재생해보면 미세한 발음의 자연스러움이나 억양 표현 등에서 고급 모델의 품질을 느낄 수 있습니다. 다만 요청 처리 시간이 약간 더 걸릴 수 있다는 점을 감안하세요.

>참고: OpenAI TTS의 요금은 입력 문자 수를 기준으로 부과되며, 1000자당 \$0.015부터 시작합니다. 고품질 모델을 쓴다고 해서 추가 요금이 부과되는지는 공식 가격 정책을 참고해야 하지만, 일반적으로 더 많은 연산을 수행하므로 속도 면에서만 차이가 있습니다.

3.3 다국어 텍스트 변환하기

OpenAI의 TTS는 여러 언어를 지원합니다. 기본적인 목소리들은 영어 화자에 최적화되어 있지만, Whisper 모델의 언어 지원을 기반으로 다양한 언어의 텍스트를 발음할 수 있습니다.

예를 들어 한국어, 스페인어, 프랑스어 등의 텍스트를 입력하면 해당 언어로 발음된 음성을 얻을 수 있습니다. 이번에는 한국어 문장을 입력으로 주어 음성으로 변환해보겠습니다. 목소리는 shimmer를 사용하고, 속도는 약간 느리게 (speed=0.8) 설정해 보겠습니다:

이 코드는 한국어 문장을 받아 Shimmer 목소리로 음성을 생성합니다. speed=0.8로 설정하여 기본 속도의 80%로 조금 느리게 발음되도록 했습니다. 이렇게 하면 한국어 발음이 너무 빠르지 않게 출력되어 듣기 더 편할 수 있습니다.

OpenAI TTS는 Whisper 모델의 다국어 지원을 따르므로, 한국어 뿐만 아니라 여러 언어를 지원합니다. Whisper 모델이 잘 동작하는 언어들은 TTS에서도 무리 없이 사용 가능하며, 예를 들면 스페인어 문장을 넣으면 스페인어 발음의 음성이 나옵니다. 다만 현재의 목소리들은 주로 영어 억양 기반이라서, 다른 언어를 발음하더라도 약간의 억양 특징이 섞일 수 있습니다. 그럼에도 불구하고 한국어 등 꽤 자연스럽게 들리는 것을 확인할 수 있습니다.

>팁: 만약 특정 언어로 발음된 음성을 얻을 때 발음이 어색하다면, 문자를 발음 나는 대로 표기하거나, 문장 부호로 억양을 조절해보는 방법도 있습니다. 예를 들어 영어 문장에서도 쉼표, 마침표 등을 넣으면 잠시 멈춤이 생겨 더 자연스럽게 들릴 수 있습니다. TTS 시스템은 입력 텍스트를 그대로 읽기 때문에, 철자가 아닌 발음을 유도하려면 입력을 조정하는 트릭을 쓸 수도 있습니다.

3.4 발음 속도 조절하기

앞서 한국어 예제에서 speed 파라미터를 사용했듯이, TTS API는 발음 속도를 조절할 수 있는 옵션을 제공합니다. speed 값을 0.25배에서 4배까지 조절할 수 있으며, 기본값은 1.0 (보통 속도)입니다

- speed=1.0 : 기본 속도 (보통 사람 말하기 속도)
- speed<1.0 : 느린 발음 (예: 0.5는 절반 속도)
- speed>1.0 : 빠른 발음 (예: 2.0은 두 배 속도로 빨리 말함)

이미 speed=0.8과 같이 느리게 하는 예제를 살펴봤으니, 하나 더 예를 들어 빠르게 말하기를 시도해보겠습니다:

위 코드에서는 speed=2.0으로 설정하여 Fable 목소리가 평소보다 두 배 빠르게 말하도록 했습니다. 나중에 이 오디오를 들어보면 같은 문장이라도 매우 빠르게 읽히는 것을 확인할 수 있습니다. 이처럼 상황에 따라 속도를 조절하면 긴 문장을 짧은 시간에 듣거나, 어린아이에게 들려주듯 천천히 말하게 하는 등 다양한 활용이 가능합니다.

지금까지 OpenAI TTS의 여러 기능을 사용하여 텍스트를 음성으로 변환하는 방법을 살펴보았습니다. 마지막으로, 이렇게 생성된 음성 데이터를 파일로 저장하고 실제로 들어보는 방법을 알아보겠습니다.

4. 오디오 파일 저장 및 Jupyter에서 재생하기

TTS API로부터 받은 음성 데이터는 메모리 상에서 바이너리 데이터(bytes) 형태로 존재합니다. 이 데이터를 파일로 저장하면 일반적인 오디오 플레이어로 재생할 수 있는 파일(예: MP3)을 얻을 수 있고, Jupyter Notebook에서는 해당 파일을 불러와 오디오 위젯으로 재생할 수도 있습니다.

4.1 오디오 데이터 파일로 저장하기

OpenAI의 TTS 응답은 기본적으로 MP3 포맷의 오디오 데이터를 반환합니다 (응답 형식을 지정하지 않은 경우). 따라서 단순히 바이너리를 .mp3 확장자의 파일로 저장하면 됩니다. 위에서 한 예제의 response.content를 이용해 파일로 저장해보겠습니다. (response_fast 객체에 담긴 빠른 속도의 음성을 저장한다고 가정하겠습니다.)

파일 저장은 일반적인 파일 쓰기 방식과 동일합니다. with open("파일경로", "wb") as f:로 바이너리

쓰기 모드("wb")로 파일을 열고, `f.write`(오디오데이터)로 내용을 기록합니다. 이렇게 하면 현재 디렉토리에 `output_fast.mp3`라는 파일이 생성되고, 변환된 음성이 그 안에 저장됩니다. OpenAI Python 라이브러리는 편의를 위해 스트리밍 저장 함수도 제공합니다. `response_fast`와 같은 응답 객체는 `stream_to_file("경로")`라는 메소드를 갖고 있어서 한 줄로 바로 파일 저장을 할 수도 있습니다. 예를 들어 `response_fast.stream_to_file("output_fast.mp3")`라고 하면 같은 결과를 얻습니다. 이 방법은 내부적으로 데이터를 조금씩 쓰는 스트림 방식을 사용하여 메모리 효율성을 높일 수 있습니다.

>참고: OpenAI TTS API는 기본 mp3 외에도 원하면 다른 오디오 포맷으로 받을 수 있습니다. `response_format` 파라미터에 "wav", "aac", "flac" 등 지원되는 포맷을 지정하면 해당 형식으로 반환받을 수 있습니다. 예를 들어 `openai_client.audio.speech.create(..., response_format="wav")`로 요청하면 `response.content`가 WAV 포맷의 데이터로 구성됩니다. 일반적인 사용에서는 mp3로 충분하겠지만, 필요에 따라 포맷을 선택하세요.

4.2 Jupyter Notebook에서 오디오 재생하기

이제 저장된 오디오 파일을 재생해볼 차례입니다. Jupyter Notebook에서는 `IPython.display` 모듈의 `Audio` 클래스를 이용하면 노트북 상에서 직접 오디오를 들을 수 있습니다. 이 기능은 브라우저 상에 오디오 플레이어 위젯을 나타나게 하고, 오디오를 재생할 수 있도록 도와줍니다.

다음 코드는 앞서 저장한 `output_fast.mp3` 파일을 불러와 재생하는 예시입니다:

위 코드를 실행하면 Jupyter Notebook 출력 영역에 오디오 컨트롤 바가 나타날 것입니다. `autoplay=True`로 설정했으므로 자동으로 재생도 시작됩니다. (브라우저 정책에 따라 자동 재생이 막힐 수 있는데, 그 경우 수동으로 재생 버튼을 누르면 됩니다.) 이제 TTS로 생성한 음성을 실제로 들어볼 수 있습니다.

만약 메모리의 데이터로 바로 재생하고 싶다면, `Audio(data=audio_bytes, rate=...)` 형태로도 가능합니다. 그러나 TTS의 결과는 이미 압축된 오디오 파일 형태이므로, `filename`을 통해 재생하는 편이 간편합니다. 여러 개의 음성을 비교하고 싶다면 다른 파일명으로 저장한 후 각각 `Audio(...)`로 불러와 보면 됩니다.

마지막으로, Jupyter가 아닌 환경(예: 백엔드 서버)에서 TTS를 활용하는 경우에는 이렇게 파일로

저장한 후, 사용자에게 그 파일을 제공하거나 스트리밍 전송하여 들려줄 수 있습니다.